오픈소스SW의 이해

응용 모의퀴즈
선택 적용·원격 협업

CHAPTER 5 + 실습 | Git 치트시트 확장 포함

75문항 · 15문항 × 5회

문제 15개를 모두 푼 뒤, 이어지는 정답·해설로 채점하세요. 각 회차는 문제 → 정답 순서입니다.

1회 Cherry-pick 범위·의존성

문제 2쪽 / 정답 7쪽 점수 _____ / 30

2회 원격 연결·추적·동기화

문제 12쪽 / 정답 17쪽 점수 _____ / 30

3회 PR·공동 브랜치·팀 운영

문제 22쪽 / 정답 27쪽 점수 _____ / 30

4회 치트시트 확장: 상태·이력

문제 32쪽 / 정답 37쪽 점수 _____ / 30

5회 선택 적용·복구·원격 종합

문제 42쪽 / 정답 47쪽 점수 _____ / 30

자가채점 안내

문항당 2점, 총 150점입니다. 객관식·복수선택·순서·집합형은 전체 일치 시 2점입니다. OX형은 판정 1점 + 근거·수정 1점이며, 나머지는 정답에 표시한 배점으로 채점하세요. 뜻이 같은 표현과 동등한 명령도 인정합니다.

이름 _____ 총점 _____ / 150

01회 문제

Cherry-pick 범위·의존성 | 1 / 5

01 | 객관식 · 2점

feature에는 버그 수정 B와 미완성 기능 C가 있다. main에는 B의 수정만 필요하다. 가장 직접적인 선택은?

① feature 전체 merge
② B를 cherry-pick
③ 원격 연결 삭제
④ main 강제 삭제

답 / 풀이

02 | 명령작성 · 2점

현재 develop에 있고, main에 커밋 a1b2c3의 변경을 가져오려 한다. 작업트리는 깨끗하다. 필요한 두 명령을 순서대로 쓰시오.

답 / 풀이

03 | 이력추론 · 2점

main은 A→M, feature는 A→B다. main에서 B를 기본 cherry-pick해 새 커밋 B'를 만들었다. B'의 부모와 feature의 끝을 쓰시오.

답 / 풀이

01회 문제

Cherry-pick 범위·의존성 | 2 / 5

04 | OX·수정 · 2점

“특정 커밋을 cherry-pick하면 그 브랜치의 뒤쪽 커밋까지 자동으로 모두 가져온다.” O/X와 적용 범위를 쓰시오.

답 / 풀이

05 | 의존성 분석 · 2점

feature의 B는 helper 함수를 추가하고, C는 helper를 호출한다. main에는 helper가 없다. C만 충돌 없이 cherry-pick했다. 발생 가능한 기능 오류와 대응을 쓰시오.

답 / 풀이

06 | 차이해석 · 2점

feature 커밋 B에서 x=1을 x=2로 바꾸고, C에서는 별도 y=10을 y=20으로 바꿨다. main은 x=1,y=10이다. C만 성공적으로 cherry-pick한 뒤 x,y는?

답 / 풀이

01회 문제

Cherry-pick 범위·의존성 | 3 / 5

07 | 조건판정 · 2점

main이 커밋 B의 변경과 같은 줄을 다른 내용으로 바꿨다. B를 cherry-pick하면 항상 자동 성공하는가? 판단과 다음 절차를 쓰시오.

답 / 풀이

08 | 명령선택 · 2점

진행 중인 cherry-pick에서 충돌이 났다. 시작 전에는 작업트리가 깨끗했다. 이번 시도를 포기하고 시작 전 상태로 돌아가려면 어떤 명령을 쓰는가?

답 / 풀이

09 | 연결형 · 2점

A: 최신 개발 버그 수정을 이전 유지보수 버전에 적용. B: 미완성 기능과 분리해 긴급 수정만 배포. 강의의 용어와 연결하라.

답 / 풀이

01회 문제

Cherry-pick 범위·의존성 | 4 / 5

10 | 오류수정 · 2점

main으로 버그 수정을 옮기려다 feature에서 git cherry-pick H를 실행했다. 아직 명령 결과를 따지기 전, 잘못된 전제와 올바른 대상 선택을 쓰시오.

답 / 풀이

11 | 이력비교 · 2점

같은 기능 수정이 B와 B'라는 서로 다른 커밋으로 각 브랜치에 있다. B'는 B를 cherry-pick한 결과라고 주어졌다. ID가 다르다는 것과 변경 내용의 관계를 설명하라.

답 / 풀이

12 | 선택설계 · 2점

한 커밋에 버그 수정과 대형 UI 개편이 섞였다. 버그만 안전하게 다른 버전에 적용하려 한다. cherry-pick 한 번이 목표에 맞지 않을 수 있는 이유와 개발 습관을 쓰시오.

답 / 풀이

01회 문제

Cherry-pick 범위·의존성 | 5 / 5

13 | 결과추적 · 2점

main의 title='학교', color='blue'. feature의 단일 커밋 F는 color만 green으로 바꾼다. main에서는 title을 '축제'로 바꾼 뒤 F를 적용했고 충돌은 없다. 최종 두 값은?

답 / 풀이

14 | 복수선택 · 2점

cherry-pick 전에 검토할 항목을 모두 고르시오.
① 받는 브랜치
② 커밋이 실제로 바꾸는 내용
③ 선행 코드 의존성
④ 작성자 이름이 같으면 무조건 호환이라는 가정

답 / 풀이

15 | 통합판단 · 2점

수정 B를 release에 cherry-pick한 뒤 단위 시험만 통과했다. 이전 버전에서는 데이터 형식이 다르다. 추가 위험과 검증 방향을 쓰시오.

답 / 풀이

01회 정답·해설

Cherry-pick 범위·의존성 | 1 / 5

01 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

②

해설

개별 커밋의 변경을 선택적으로 적용하는 용도다. C까지 도입하려는 상황이 아니다.

근거 5장 p.3, 5-7

02 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git switch main (1점). git cherry-pick a1b2c3 (1점).

해설

변경은 현재 브랜치에 적용되므로 받는 쪽부터 선택한다.

근거 5장 p.7 기반 응용

03 | 이력추론 · 내 점수 ___ / 2

정답 · 채점 기준

B'의 부모는 M (1점). feature는 B 유지 (1점).

해설

원본 B를 main의 부모로 붙이는 merge와 다르다. 현재 이력 위에 변경을 다시 기록한다.

근거 5장 p.5-7; 보완 C

01회 정답·해설

Cherry-pick 범위·의존성 | 2 / 5

04 | OX·수정 · 내 점수 ___ / 2

정답 · 채점 기준

X (1점). 지정한 커밋의 변경을 적용하며 이후 커밋은 자동 포함되지 않음 (1점).

해설

브랜치 전체의 끝을 병합하는 것과 커밋 하나를 고르는 것을 구분한다.

근거 5장 p.5-8

05 | 의존성 분석 · 내 점수 ___ / 2

정답 · 채점 기준

helper 미정의 등 의존성 누락 (1점). 필요한 선행 변경을 함께 검토·도입하거나 C를 독립적으로 수정 (1점).

해설

텍스트 적용 성공은 실행 의존성 충족을 보장하지 않는다.

근거 5장 p.3, 7 기반 응용

06 | 차이해석 · 내 점수 ___ / 2

정답 · 채점 기준

x=1 (1점). y=20 (1점).

해설

C 시점 전체 파일 스냅샷으로 덮는 것이 아니라 C가 도입한 변경을 적용한다.

근거 5장 p.5-7; 보완 C

01회 정답·해설

Cherry-pick 범위·의존성 | 3 / 5

07 | 조건판정 · 내 점수 ___ / 2

정답 · 채점 기준

항상 성공하지 않으며 충돌 가능 (1점). 충돌 내용을 해결·add하고 cherry-pick --continue (1점).

해설

개별 커밋 적용에도 문맥이나 내용 충돌이 생길 수 있다.

근거 5장 p.7 기반 응용; 보완 C

08 | 명령선택 · 내 점수 ___ / 2

정답 · 채점 기준

git cherry-pick --abort

해설

merge --abort가 아니라 진행 중인 작업 종류에 맞는 명령을 사용한다.

근거 5장 p.7 기반 응용; 보완 C

09 | 연결형 · 내 점수 ___ / 2

정답 · 채점 기준

A-Backporting (1점). B-Hot Fix·격리 변경 (1점).

해설

모든 새 기능을 합치지 않고 필요한 수정만 옮기는 목적이다.

근거 5장 p.3

01회 정답·해설

Cherry-pick 범위·의존성 | 4 / 5

10 | 오류수정 · 내 점수 ___ / 2

정답 · 채점 기준

현재 브랜치가 받는 쪽이라는 점을 놓침 (1점). main으로 전환한 뒤 적용 계획 실행 (1점).

해설

커밋 인수는 적용할 변경이지 목적 브랜치가 아니다.

근거 5장 p.7 기반 응용

11 | 이력비교 · 내 점수 ___ / 2

정답 · 채점 기준

다른 부모·기록으로 커밋 ID가 달라질 수 있음 (1점). 적용한 변경의 의미는 같을 수 있음 (1점).

해설

커밋 정체성과 코드 변경의 의미는 별도 비교 대상이다.

근거 5장 p.5-7; 보완 C

12 | 선택설계 · 내 점수 ___ / 2

정답 · 채점 기준

한 커밋의 두 변경이 함께 적용될 수 있음 (1점). 독립 목적별 작은 커밋으로 변경을 분리 (1점).

해설

선택 단위를 잘 나누어 두어야 이후 선택 적용이 쉬워진다.

근거 5장 p.3, 7 기반 응용

01회 정답·해설

Cherry-pick 범위·의존성 | 5 / 5

13 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

title='축제' (1점). color='green' (1점).

해설

현재 브랜치의 독립 변경을 유지하면서 선택한 변경을 더한다.

근거 5장 p.5-7 기반 응용

14 | 복수선택 · 내 점수 ___ / 2

정답 · 채점 기준

①, ②, ③

해설

브랜치·변경 범위·의존성을 함께 봐야 선택 적용의 의도와 결과가 맞는다.

근거 5장 p.3, 7 기반 응용

15 | 통합판단 · 내 점수 ___ / 2

정답 · 채점 기준

이전 버전 데이터·인터페이스와 호환되지 않을 수 있음 (1점). release 환경의 실제 입력·연결 기능으로 통합 시험 (1점).

해설

Backport는 커밋을 옮기는 작업과 대상 버전에서의 검증을 함께 요구한다.

근거 5장 p.3 기반 응용

02회 문제

원격 연결·추적·동기화 | 1 / 5

16 | 객관식 · 2점

로컬 저장소에 origin 주소를 등록했다. 그 즉시 일어나지 않는 것은?
① 원격 이름 설정
② URL 연결 정보 저장
③ 로컬 커밋의 원격 자동 업로드
④ 이후 원격 명령에서 origin 사용

답 / 풀이

17 | 명령작성 · 2점

이미 존재하는 로컬 저장소에 원격 URL을 origin으로 등록하고 확인하라. URL은 https://example.com/team/repo.git이다. 두 명령을 쓰시오.

답 / 풀이

18 | 방향판단 · 2점

지역·원격 저장소 사이에서 A는 지역 커밋을 보내기, B는 원격 변경을 가져와 현재 브랜치에 통합하기이다. 일반적인 작업 이름을 연결하라.

답 / 풀이

02회 문제

원격 연결·추적·동기화 | 2 / 5

19 | 빈칸 · 2점

원격 별칭 origin은 일반적으로 쓰는 (A)이며, 로컬 브랜치 main과 같은 개념은 (B: 맞다/아니다).

답 / 풀이

20 | 명령작성 · 2점

현재 feature/login의 로컬 커밋을 처음 원격에 올리며 같은 이름의 원격 브랜치와 추적 관계를 설정하려 한다. 명령과 -u의 역할을 쓰시오.

답 / 풀이

21 | 조건판정 · 2점

새 브랜치 test에서 기본 git push가 'no upstream branch'로 실패했다. 원격 주소 origin은 올바르다. 필요한 수정과, 주소를 다시 등록할 필요가 없는 이유를 쓰시오.

답 / 풀이

02회 문제

원격 연결·추적·동기화 | 3 / 5

22 | OX·수정 · 2점

“원격 저장소는 반드시 GitHub에 있어야 하므로 다른 서버의 Git 저장소는 remote가 될 수 없다.” O/X와 의미를 쓰시오.

답 / 풀이

23 | 결과추적 · 2점

local main=A, origin/main=A다. 원격 main은 팀원 커밋 B까지 진행했다. git fetch origin에 성공했다. local main과 origin/main의 위치를 쓰시오.

답 / 풀이

24 | 명령비교 · 2점

현재 main에서 원격 main을 가져와 merge 방식으로 통합하려 한다. pull 설정에 의존하지 않도록 명시하는 명령과, fetch만 실행했을 때의 차이를 쓰시오.

답 / 풀이

02회 문제

원격 연결·추적·동기화 | 4 / 5

25 | 원격진단 · 2점

git remote -v가 출력된다는 사실만으로 push 권한·인증·서버 접속이 모두 성공했다고 할 수 있는가? 이유와 추가 확인을 쓰시오.

답 / 풀이

26 | 충돌상황 · 2점

로컬 main과 원격 main이 A에서 서로 다른 커밋 L,R로 진행했다. 기본 push가 non-fast-forward로 거부됐다. 강제 덮어쓰기 전에 필요한 두 단계는?

답 / 풀이

27 | 파일구분 · 2점

README.md, .gitignore, LICENSE를 각각 실행·프로젝트 설명 / 추적 제외 규칙 / 이용허락 조건과 연결하라.

답 / 풀이

02회 문제

원격 연결·추적·동기화 | 5 / 5

28 | 연결삭제 · 2점

git remote remove origin을 실행했다. 원격 GitHub 저장소와 로컬 프로젝트 파일이 함께 지워지는가? 실제 제거 대상도 쓰시오.

답 / 풀이

29 | 비교서술 · 2점

서버의 기존 이력을 새 폴더에 처음 받아오려는 A, 이미 복제한 저장소의 최신 변경을 확인하려는 B가 있다. 적절한 명령 종류를 연결하라.

답 / 풀이

30 | 통합추적 · 2점

로컬에서 hello.py를 저장했지만 add·commit은 하지 않았다. git push -u origin main은 기존 커밋만 성공적으로 올렸다. hello.py가 원격에 없는 이유와 필요한 흐름을 쓰시오.

답 / 풀이

02회 정답·해설

원격 연결·추적·동기화 | 1 / 5

16 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

③

해설

remote add는 연결 설정이며 push가 아니다.

근거 5장 p.20-24, 28

17 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git remote add origin https://example.com/team/repo.git (1점). git remote -v (1점).

해설

예시 주소는 문법 연습용이다. 원격 생성·인증이 자동 완료되는 명령은 아니다.

근거 5장 p.28

18 | 방향판단 · 내 점수 ___ / 2

정답 · 채점 기준

A-push (1점). B-pull (1점).

해설

fetch는 가져오기만 하고 현재 브랜치 통합은 하지 않는다는 차이도 구분한다.

근거 5장 p.14, 21-24; 치트 p.1

02회 정답·해설

원격 연결·추적·동기화 | 2 / 5

19 | 빈칸 · 내 점수 ___ / 2

정답 · 채점 기준

A: 원격 저장소 이름 (1점). B: 아니다 (1점).

해설

git push -u origin main의 두 인수가 서로 다른 대상을 뜻한다.

근거 5장 p.28, 31

20 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git push -u origin feature/login (1점). 기본 upstream 추적 관계 설정 (1점).

해설

처음 push 이후 매번 원격·브랜치를 명시하는 수고를 줄일 수 있다.

근거 5장 p.31-33

21 | 조건판정 · 내 점수 ___ / 2

정답 · 채점 기준

git push -u origin test (1점). 원격 URL 문제가 아니라 현재 브랜치의 추적 관계 미설정 (1점)
.

해설

서로 다른 설정 오류를 구분해야 한다.

근거 5장 실습 p.9

02회 정답·해설

원격 연결·추적·동기화 | 3 / 5

22 | OX·수정 · 내 점수 ___ / 2

정답 · 채점 기준

X (1점). GitHub는 원격 호스팅 서비스 중 하나이며 다른 서버의 저장소도 가능 (1점).

해설

강의의 GitHub 사례를 원격 저장소 전체의 정의로 확대하지 않는다.

근거 5장 p.11-14

23 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

local main=A (1점). origin/main=B (1점).

해설

fetch는 원격 추적 정보를 갱신하지만 현재 브랜치를 자동 병합하지 않는다.

근거 치트 p.1

24 | 명령비교 · 내 점수 ___ / 2

정답 · 채점 기준

git pull --no-rebase origin main (1점). fetch만 하면 현재 main에 자동 통합하지 않음 (1점)
.

해설

pull의 통합 방식은 설정에 따라 달라질 수 있어 이 문제는 명시적 옵션을 사용한다.

근거 5장 p.29; 치트 p.1

02회 정답·해설

원격 연결·추적·동기화 | 4 / 5

25 | 원격진단 · 내 점수 ___ / 2

정답 · 채점 기준

아니다, 로컬 연결 설정을 보여 줄 뿐 (1점). 실제 인증·권한과 원격 통신 결과 확인 (1점).

해설

설정된 URL과 접근 가능한 저장소는 같은 증거가 아니다.

근거 5장 p.28, 31 기반 응용

26 | 충돌상황 · 내 점수 ___ / 2

정답 · 채점 기준

원격 이력을 가져와 확인 (1점). 팀 변경과 로컬 변경을 통합·검증한 뒤 push (1점).

해설

먼저 올라온 다른 사람의 변경을 버리지 않고 반영하는 흐름이다.

근거 5장 p.14, 39; 치트 p.1 기반 응용

27 | 파일구분 · 내 점수 ___ / 2

정답 · 채점 기준

README.md-설명 / .gitignore-추적 제외 규칙 / LICENSE-이용허락 조건

해설

전부 맞으면 2점. 세 파일은 원격 초기화에서 함께 등장해도 목적이 다르다.

근거 5장 p.16, 20; 치트 p.1

02회 정답·해설

원격 연결·추적·동기화 | 5 / 5

28 | 연결삭제 · 내 점수 ___ / 2

정답 · 채점 기준

둘 다 자동 삭제되지 않음 (1점). 로컬의 origin 연결 설정 제거 (1점).

해설

연결 정보 삭제와 저장소 자체 삭제를 구분한다.

근거 5장 p.18, 28; 치트 p.1

29 | 비교서술 · 내 점수 ___ / 2

정답 · 채점 기준

A: git clone URL (1점). B: git fetch origin 등 기존 저장소 갱신 (1점).

해설

clone을 매번 할 필요는 없다. B의 목적이 확인만이면 fetch 후 검토가 적절하다.

근거 5장 p.39; 치트 p.1

30 | 통합추적 · 내 점수 ___ / 2

정답 · 채점 기준

저장만 한 변경은 push할 커밋에 포함되지 않음 (1점). add→commit→push (1점).

해설

파일 저장, 로컬 기록, 원격 공유는 별도 단계다.

근거 5장 p.23-24, 31-32; 치트 p.1

03회 문제

PR·공동 브랜치·팀 운영 | 1 / 5

31 | 객관식 · 2점

팀원이 feature를 push한 뒤 “main 통합이 끝났다”고 말했다. 일반적으로 추가로 필요한 과정은?
① 검토·병합
② 로컬 폴더 이름 변경
③ Git 삭제
④ 원격 URL 공개만

답 / 풀이

32 | PR방향 · 2점

feature의 변경을 main에 반영하는 Pull Request를 만든다. base와 compare에 각각 선택할 브랜치를 쓰시오.

답 / 풀이

33 | 순서배열 · 2점

A 팀원이 clone·자기 브랜치 작업 / B 팀장이 저장소 생성·협업자 추가 / C 초기 README 등 커밋 / D 변경 게시·병합 요청 / E 검토 후 병합. 강의 흐름으로 배열하라.

답 / 풀이

03회 문제

PR·공동 브랜치·팀 운영 | 2 / 5

34 | 권한구별 · 2점

팀 저장소가 public이다. 외부인이 볼 수 있다는 사실만으로 누구나 원격 main에 직접 push할 수 있는가? 강의의 협업 설정과 연결해 답하라.

답 / 풀이

35 | 협업분석 · 2점

실습에서 한 브랜치에 두 명이 작업하도록 했다. 왜 push·pull 순서 조율과 충돌 검토가 중요한지 두 관점에서 쓰시오.

답 / 풀이

36 | OX·수정 · 2점

“Pull Request와 git pull은 이름이 비슷하므로 같은 명령이다.” O/X와 차이를 쓰시오.

답 / 풀이

03회 문제

PR·공동 브랜치·팀 운영 | 3 / 5

37 | 문서설계 · 2점

README에는 학번·GitHub ID만 있다. 실습의 역할 분담 확인에 추가로 필요한 기록과, 챕터별 산출물 구조를 쓰시오.

답 / 풀이

38 | 이름변경 · 2점

현재 로컬 저장소의 master를 main으로 이름 변경하려 한다. 강의의 명령과 이 명령만으로 원격 브랜치까지 자동 변경되는지 쓰시오.

답 / 풀이

39 | 병합후 갱신 · 2점

GitHub에서 PR이 main에 병합됐지만 내 로컬 main은 예전 커밋이다. 로컬 작업을 최신 통합 결과로 맞추는 명령 흐름을 쓰시오. 작업트리는 깨끗하다.

답 / 풀이

03회 문제

PR·공동 브랜치·팀 운영 | 4 / 5

40 | 코드리뷰 · 2점

PR에 “No conflicts”가 나타났다. 팀장이 코드 검토를 생략해도 된다고 주장한다. 보장되는 범위와 추가 검토를 쓰시오.

답 / 풀이

41 | 명령작성 · 2점

main에 test를 로컬에서 검토·병합한 뒤 원격 main에도 반영하려 한다. main에서 병합은 완료됐다. 마지막 공유 명령과 방향을 쓰시오.

답 / 풀이

42 | 상태구분 · 2점

git branch에는 main,test가 있고 git branch -r에는 origin/main만 보인다. 최신 fetch를 했으며 원격에 test는 없다. 로컬 test를 다른 팀원이 받게 할 명령을 쓰시오.

답 / 풀이

03회 문제

PR·공동 브랜치·팀 운영 | 5 / 5

43 | 기록분석 · 2점

팀원은 자신이 맡은 문제를 별도 파일에 풀었지만 README 분담 기록은 다른 사람 이름이다. 코드 병합 성공과 별개로 수정할 사항과 이유를 쓰시오.

답 / 풀이

44 | 오류진단 · 2점

팀장이 본인도 main에서 직접 새 기능을 계속 개발한다. 강의의 팀 실습 흐름에서 다른 팀원과 맞춰야 할 작업 방식과 이유를 쓰시오.

답 / 풀이

45 | 통합상황 · 2점

두 팀원이 같은 원격 test를 A에서 시작했다. 갑이 B를 push한 뒤 을이 독립 커밋 C를 push하려 한다. 예상 문제와 보존해야 할 이력을 쓰시오.

답 / 풀이

03회 정답·해설

PR·공동 브랜치·팀 운영 | 1 / 5

31 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

①

해설

기능 브랜치를 게시한 것과 main에 반영한 것은 다르다.

근거 5장 p.39; 5장 실습 p.11-14

32 | PR방향 · 내 점수 ___ / 2

정답 · 채점 기준

base=main (1점). compare=feature (1점).

해설

받는 기준 브랜치와 변경을 가져올 브랜치의 방향을 구분한다.

근거 5장 실습 p.12-14

33 | 순서배열 · 내 점수 ___ / 2

정답 · 채점 기준

B → C → A → D → E

해설

초기 이력을 기준으로 복제하고 각자 작업한 뒤 검토를 거쳐 통합한다.

근거 5장 p.39; 5장 실습 p.7

03회 정답·해설

PR·공동 브랜치·팀 운영 | 2 / 5

34 | 권한구별 · 내 점수 ___ / 2

정답 · 채점 기준

아니다 (1점). 공개 열람과 쓰기 권한은 별개이며 collaborator 등 권한 설정 필요 (1점).

해설

public이라는 속성은 임의 쓰기 허용과 같은 뜻이 아니다.

근거 5장 p.39; 5장 실습 p.7

35 | 협업분석 · 내 점수 ___ / 2

정답 · 채점 기준

동일 원격 브랜치가 상대 커밋으로 먼저 진행할 수 있음 (1점). 같은 내용 수정 시 통합·충돌 해결이 필요 (1점).

해설

브랜치가 같다고 자동으로 두 사람의 로컬 상태가 같아지는 것은 아니다.

근거 5장 실습 p.7

36 | OX·수정 · 내 점수 ___ / 2

정답 · 채점 기준

X (1점). PR은 변경 검토·병합 요청, git pull은 원격 변경을 가져와 로컬에 통합하는 Git 명령 (1점).

해설

플랫폼의 협업 절차와 로컬 저장소 동기화 명령을 구분한다.

근거 5장 p.12, 14; 5장 실습 p.12-15

03회 정답·해설

PR·공동 브랜치·팀 운영 | 3 / 5

37 | 문서설계 · 내 점수 ___ / 2

정답 · 채점 기준

GitHub ID별 분담 문제 기록 (1점). chapter-branches.md 등 챕터별 문서 생성 (1점).

해설

구성원 명단만으로 누가 어떤 문제를 맡았는지 알 수 없다.

근거 5장 실습 p.7

38 | 이름변경 · 내 점수 ___ / 2

정답 · 채점 기준

git branch -m master main (1점). 원격 이름까지 자동 변경하지 않음 (1점).

해설

로컬 참조 이름 변경과 원격 상태 갱신은 구분한다.

근거 5장 실습 p.7; 치트 p.1

39 | 병합후 갱신 · 내 점수 ___ / 2

정답 · 채점 기준

git switch main (1점). git pull --no-rebase origin main 등 적절한 갱신 (1점).

해설

서버의 병합이 모든 개발자의 로컬 브랜치를 즉시 바꾸는 것은 아니다.

근거 5장 실습 p.14-15; 치트 p.1

03회 정답·해설

PR·공동 브랜치·팀 운영 | 4 / 5

40 | 코드리뷰 · 내 점수 ___ / 2

정답 · 채점 기준

텍스트 병합 가능 여부의 신호일 뿐 기능 정답 보장은 아님 (1점). 요구사항·동작·변경 범위 검토 및 테스트 (1점).

해설

충돌 없음과 품질 승인 완료를 혼동하지 않는다.

근거 5장 p.12, 39; 5장 실습 p.13

41 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git push origin main (1점). 로컬 main 커밋을 원격 origin의 main으로 전송 (1점).

해설

로컬 병합 성공만으로 서버가 갱신되지는 않는다.

근거 5장 실습 p.15

42 | 상태구분 · 내 점수 ___ / 2

정답 · 채점 기준

git push -u origin test

해설

로컬 브랜치 생성과 원격 게시를 구분한다. 실제 push 권한은 있다고 가정한다.

근거 5장 p.33; 치트 p.1

03회 정답·해설

PR·공동 브랜치·팀 운영 | 5 / 5

43 | 기록분석 · 내 점수 ___ / 2

정답 · 채점 기준

실제 담당자·GitHub ID와 문제 분담 기록 수정 (1점). 작업 출처와 검토 책임을 정확히 확인 가능하게 함 (1점).

해설

협업 문서의 일치도 과제 수행의 일부다.

근거 5장 실습 p.7

44 | 오류진단 · 내 점수 ___ / 2

정답 · 채점 기준

팀장도 별도 작업 브랜치 사용 (1점). 검토·통합 경로와 안정적인 main을 유지 (1점).

해설

역할이 팀장이라는 사실이 작업 분리 원칙을 없애지 않는다.

근거 5장 p.39

45 | 통합상황 · 내 점수 ___ / 2

정답 · 채점 기준

비 fast-forward 거부가 발생할 수 있음 (1점). B와 C 양쪽 변경을 검토·통합하여 보존 (1점).

해설

같은 브랜치 이름을 쓴다고 마지막 push가 앞 변경을 안전하게 덮는 것이 아니다.

근거 5장 실습 p.7; 치트 p.1

04회 문제

치트시트 확장: 상태·이력 | 1 / 5

46 | 상태추적 · 2점

[치트시트 확장] 추적 파일의 HEAD=A, index=B, 작업트리=C다. git restore --staged file 후 index와 작업트리 내용은? 기본 소스는 HEAD다.

답 / 풀이

47 | 상태추적 · 2점

[치트시트 확장] HEAD=A, index=B, 작업트리=C다. git restore file을 기본 옵션으로 실행했다. index와 작업트리 내용은?

답 / 풀이

48 | 비교형 · 2점

[치트시트 확장] HEAD=A, index=B, 작업트리=C다. git diff와 git diff --cached는 각각 어떤 두 상태를 비교하는가?

답 / 풀이

04회 문제

치트시트 확장: 상태·이력 | 2 / 5

49 | 결과추적 · 2점

[치트시트 확장] A→B, 현재 HEAD=B이고 index·작업트리도 B다. git reset --soft HEAD~1 후 HEAD·index·작업트리 상태를 쓰시오.

답 / 풀이

50 | 결과추적 · 2점

[치트시트 확장] A→B이며 세 상태가 모두 B다. git reset --mixed HEAD~1 후 index·작업트리를 쓰시오.

답 / 풀이

51 | 객관식 · 2점

[치트시트 확장] 공유된 커밋의 변경을 취소하되 기존 이력을 남겨야 한다. 적합한 기본 선택은?

① git revert
② 무조건 reset --hard
③ 브랜치 강제 삭제
④ remote remove

답 / 풀이

04회 문제

치트시트 확장: 상태·이력 | 3 / 5

52 | 명령작성 · 2점

[치트시트 확장] 이미 추적 중인 config.local을 작업 폴더에는 남기고 이후 추적에서 빼려 한다. 인덱스에서 제거할 명령과 별도 제외 설정을 쓰시오.

답 / 풀이

53 | stash판단 · 2점

[치트시트 확장] 추적 파일 수정과 새 미추적 notes.txt가 있다. 둘 다 임시 보관하려 한다. 기본 stash push 대신 쓸 옵션과, 무시된 파일까지 포함하는 다른 옵션은?

답 / 풀이

54 | 복수선택 · 2점

[치트시트 확장] stash에 대해 옳은 것을 모두 고르시오.
① apply는 기본적으로 항목 유지
② pop은 성공하면 항목 삭제
③ pop이 충돌하면 해당 항목 유지
④ clear는 적용 없이 안전 백업을 하나 더 만듦

답 / 풀이

04회 문제

치트시트 확장: 상태·이력 | 4 / 5

55 | rebase추론 · 2점

[치트시트 확장] feature의 독립 커밋 B,C를 최신 main 위에 rebase했다. 변경된 커밋은 B',C'다. 부모 기반과 커밋 ID에 생긴 변화를 쓰시오.

답 / 풀이

56 | squash판단 · 2점

[치트시트 확장] git merge --squash feature가 성공했다. “이미 새 커밋이 생겼다”는 주장에 대한 판정과 후속 명령을 쓰시오.

답 / 풀이

57 | 옵션비교 · 2점

[치트시트 확장] fast-forward가 가능한 상황에서도 병합 이력을 별도 커밋으로 남기고 싶다. 옵션과 기본 fast-forward와의 차이를 쓰시오.

답 / 풀이

04회 문제

치트시트 확장: 상태·이력 | 5 / 5

58 | 태그공유 · 2점

[치트시트 확장] 현재 커밋에 v1.0 태그를 만들었다. 원격에 그 태그를 명시적으로 올리는 명령과, 로컬 생성만으로 원격 공유가 끝나는지 쓰시오.

답 / 풀이

59 | 분리HEAD · 2점

[치트시트 확장] git switch --detach OLD 후 새 커밋 N을 만들었다. 이 상태가 읽기 전용인지와, N을 이름 있는 작업 흐름으로 보존할 명령을 쓰시오.

답 / 풀이

60 | 작업구별 · 2점

[치트시트 확장] merge 충돌 중단, rebase 충돌 중단, cherry-pick 충돌 중단에 맞는 --abort 명령을 각각 쓰시오.

답 / 풀이

04회 정답·해설

치트시트 확장: 상태·이력 | 1 / 5

46 | 상태추적 · 내 점수 ___ / 2

정답 · 채점 기준

index=A (1점). 작업트리=C (1점).

해설

스테이징 취소는 작업트리 수정까지 버리는 명령이 아니다.

근거 치트 p.1

47 | 상태추적 · 내 점수 ___ / 2

정답 · 채점 기준

index=B (1점). 작업트리=B (1점).

해설

기본 restore의 작업트리 복원 소스는 index다. HEAD로 무조건 되돌린다고 외우지 않는다.

근거 치트 p.1

48 | 비교형 · 내 점수 ___ / 2

정답 · 채점 기준

git diff: index B와 작업트리 C (1점). --cached: HEAD A와 index B (1점).

해설

스테이징 전 변경과 준비된 변경을 나누어 본다.

근거 치트 p.1

04회 정답·해설

치트시트 확장: 상태·이력 | 2 / 5

49 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

HEAD=A (1점). index=B, 작업트리=B (1점).

해설

커밋 위치만 되돌리고 준비된 변경은 유지한다.

근거 치트 p.1

50 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

index=A (1점). 작업트리=B (1점).

해설

HEAD는 A로 이동한다. 파일 변경은 남지만 스테이징은 해제된다.

근거 치트 p.1

51 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

①

해설

revert는 상쇄하는 새 커밋을 추가한다. 공유 이력 자체를 뒤로 옮기는 reset과 다르다.

근거 치트 p.1

04회 정답·해설

치트시트 확장: 상태·이력 | 3 / 5

52 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git rm --cached config.local (1점). .gitignore에 해당 규칙 추가하고 변경 커밋 (1점).

해설

.gitignore만 추가해도 이미 추적 중인 파일이 자동 제거되지는 않는다.

근거 치트 p.1

53 | stash판단 · 내 점수 ___ / 2

정답 · 채점 기준

미추적 포함: -u (1점). 무시된 파일까지 포함: -a (1점).

해설

보관 범위를 구분해야 파일이 남거나 누락되는 이유를 설명할 수 있다.

근거 치트 p.1

54 | 복수선택 · 내 점수 ___ / 2

정답 · 채점 기준

①, ②, ③

해설

clear는 보관 목록 삭제이며 백업 생성이 아니다.

근거 치트 p.1

04회 정답·해설

치트시트 확장: 상태·이력 | 4 / 5

55 | rebase추론 · 내 점수 ___ / 2

정답 · 채점 기준

최신 main을 새 기준으로 재적용 (1점). 재작성된 커밋 ID 변경 (1점).

해설

내용이 비슷해도 부모와 이력이 달라질 수 있다.

근거 치트 p.1

56 | squash판단 · 내 점수 ___ / 2

정답 · 채점 기준

틀림, squash 자체는 커밋을 만들지 않음 (1점). git commit -m "..."으로 기록 (1점).

해설

변경을 합쳐 준비하는 것과 새 커밋을 만드는 단계를 구분한다.

근거 치트 p.1

57 | 옵션비교 · 내 점수 ___ / 2

정답 · 채점 기준

git merge --no-ff feature (1점). 단순 포인터 이동 대신 병합 커밋 생성 (1점).

해설

같은 변경을 합쳐도 이력의 모양을 다르게 남길 수 있다.

근거 치트 p.1

04회 정답·해설

치트시트 확장: 상태·이력 | 5 / 5

58 | 태그공유 · 내 점수 ___ / 2

정답 · 채점 기준

git push origin v1.0 (1점). 로컬 생성만으로 공유되지 않음 (1점).

해설

태그와 브랜치의 원격 전송을 구분한다. v1.0 이름 충돌은 없다고 가정한다.

근거 치트 p.1

59 | 분리HEAD · 내 점수 ___ / 2

정답 · 채점 기준

읽기 전용이 아님 (1점). N에서 git switch -c rescue 등 새 브랜치 생성 (1점).

해설

분리된 HEAD에서도 커밋할 수 있다. 다시 찾을 수 있도록 참조를 붙인다.

근거 치트 p.1

60 | 작업구별 · 내 점수 ___ / 2

정답 · 채점 기준

git merge --abort / git rebase --abort / git cherry-pick --abort

해설

모두 맞으면 2점. 진행 중인 작업 종류와 복구 명령을 일치시킨다.

근거 치트 p.1; 5장 p.7 기반 응용; 보완 C

05회 문제

선택 적용·복구·원격 종합 | 1 / 5

61 | 원격추적 · 2점

원격 main=A→B, 로컬 main=A이고 로컬 커밋은 없다. fetch 후 main이 A인 채 git merge origin/main을 실행했다. 병합 방식과 최종 로컬 main 위치는?

답 / 풀이

62 | 정렬추론 · 2점

로컬 L은 아직 공유되지 않았다. 원격은 A→R, 로컬은 A→L이다. main에서 git pull --rebase origin main이 성공했다. 새 이력 순서와 L의 ID 변화 가능성을 쓰시오.

답 / 풀이

63 | 연속상태 · 2점

[치트시트 확장] HEAD=A, index=B, 작업트리=C다. git restore --staged file 후 git restore file을 실행한다. 최종 index·작업트리는?

답 / 풀이

05회 문제

선택 적용·복구·원격 종합 | 2 / 5

64 | 의존성 선택 · 2점

커밋 B는 운영상태 필드를 추가, C는 추천에서 그 필드를 검사한다. release에는 둘 다 없다. C만 옮긴 뒤 필드 오류가 난다. 추가 검토 대상과, 재발 방지 커밋 관리 원칙을 쓰시오.

답 / 풀이

65 | 공유이력 판단 · 2점

팀원 모두 받은 main의 최근 커밋을 취소하려 한다. 로컬에서 reset --hard한 뒤 강제 push하면 생길 협업 문제와, 이력을 보존하는 대안은?

답 / 풀이

66 | 설정구별 · 2점

원격 별칭을 origin에서 upstream으로 바꾸려 한다. 로컬 브랜치 이름도 main에서 upstream으로 바꿔야 하는가? 적합한 원격 이름 변경 명령도 쓰시오.

답 / 풀이

05회 문제

선택 적용·복구·원격 종합 | 3 / 5

67 | 비밀파일 검사 · 2점

이미 커밋한 .env를 .gitignore에 추가했다. “과거 커밋에서도 자동 삭제됐다”는 주장에 대한 판정과, 현재 추적 제거가 과거 이력까지 바꾸는지 쓰시오.

답 / 풀이

68 | stash역추론 · 2점

stash pop 후 충돌했다. 사용자가 “목록에서 사라졌을 것”이라며 새로 보관하려 한다. 기본 pop의 항목 유지 여부와 먼저 확인할 명령을 쓰시오.

답 / 풀이

69 | 증거구별 · 2점

로컬 Git Graph에는 최신 커밋 C가 있지만 GitHub main에는 B만 있다. 이것만으로 C가 손실됐다고 할 수 있는가? 가능한 이유와 확인 방향을 쓰시오.

답 / 풀이

05회 문제

선택 적용·복구·원격 종합 | 4 / 5

70 | 통합명령 · 2점

feature의 커밋 H 하나만 local main에 적용하고 origin/main에 공유하려 한다. 현재 feature이며 권한·의존성·시험은 충족됐다. 세 명령을 쓰시오.

답 / 풀이

71 | 조건반전 · 2점

기존에는 신규 브랜치에 upstream이 없어 git pull이 실패했다. 지금은 upstream을 설정했다. “브랜치 인수를 생략한 pull은 항상 오류”라는 주장과 올바른 판단 기준을 쓰시오.

답 / 풀이

72 | 상태비교 · 2점

[치트시트 확장] A→B이며 모든 추적 파일 상태가 B와 같다. reset --soft HEAD~1과 reset --hard HEAD~1의 작업트리 결과를 각각 쓰시오. 다른 미커밋·미추적 파일은 없다.

답 / 풀이

05회 문제

선택 적용·복구·원격 종합 | 5 / 5

73 | PR시나리오 · 2점

PR의 compare=feature, base=main이다. 리뷰 중 feature에 수정 커밋 F를 추가하고 같은 원격 feature로 push했다. 일반적으로 기존 PR에 반영되는 변경과, 로컬 커밋만 했을 때의 차이를 쓰시오.

답 / 풀이

74 | 범위판단 · 2점

main으로 B를 cherry-pick한 뒤 feature에는 B와 C가 그대로 남았다. “원본 브랜치에서 B가 이동해 사라졌다”는 설명을 고쳐라. 원본 끝과 대상의 기록 방식을 쓰시오.

답 / 풀이

75 | 최종설계 · 2점

팀 저장소에서 필요한 버그 수정만 release에 반영하고, 공유 이력을 보존하며, 다른 팀원도 결과를 확인하게 하려 한다. 적용·검증 단계와 공유·검토 단계를 제안하라.

답 / 풀이

05회 정답·해설

선택 적용·복구·원격 종합 | 1 / 5

61 | 원격추적 · 내 점수 ___ / 2

정답 · 채점 기준

fast-forward (1점). B (1점).

해설

fetch와 현재 브랜치 통합을 두 단계로 나눠 수행한 결과다.

근거 5장 p.14; 치트 p.1

62 | 정렬추론 · 내 점수 ___ / 2

정답 · 채점 기준

A→R→L' (1점). 재적용된 L'는 ID가 달라짐 (1점).

해설

원격 변경을 기준으로 로컬 커밋을 다시 적용한 선형 이력이다.

근거 치트 p.1

63 | 연속상태 · 내 점수 ___ / 2

정답 · 채점 기준

index=A (1점). 작업트리=A (1점).

해설

첫 명령이 index를 A로 바꾸고 두 번째가 그 index를 작업트리에 복원한다.

근거 치트 p.1

05회 정답·해설

선택 적용·복구·원격 종합 | 2 / 5

64 | 의존성 선택 · 내 점수 ___ / 2

정답 · 채점 기준

B 등 선행 스키마·코드 의존성 확인 (1점). 선택 적용 시 의존관계를 기록하고 호환성 시험 (1점).

해설

한 커밋만 필요한 것처럼 보여도 데이터 구조 변경이 전제가 될 수 있다.

근거 5장 p.3, 7 기반 응용

65 | 공유이력 판단 · 내 점수 ___ / 2

정답 · 채점 기준

다른 팀원이 가진 공유 이력과 불일치·변경 소실 가능 (1점). git revert로 상쇄 커밋을 만들고 공유 (1점).

해설

공유한 이력의 재작성은 로컬 정리와 다른 영향을 준다.

근거 치트 p.1

66 | 설정구별 · 내 점수 ___ / 2

정답 · 채점 기준

그럴 필요 없음 (1점). git remote rename origin upstream (1점).

해설

원격 이름과 브랜치 이름은 서로 다른 이름 공간이다.

근거 치트 p.1

05회 정답·해설

선택 적용·복구·원격 종합 | 3 / 5

67 | 비밀파일 검사 · 내 점수 ___ / 2

정답 · 채점 기준

틀림, .gitignore는 과거 커밋을 수정하지 않음 (1점). git rm --cached 후 커밋해도 과거 이력에는 남음 (1점).

해설

추적 제외와 기존 이력 정리는 다른 문제다. 이 문항은 범위 구별을 묻는다.

근거 치트 p.1 기반 응용

68 | stash역추론 · 내 점수 ___ / 2

정답 · 채점 기준

충돌이면 해당 보관 항목 유지 (1점). git stash list 및 git status 등 확인 (1점).

해설

성공 시 삭제라는 조건을 충돌 상황까지 확대하지 않는다.

근거 치트 p.1

69 | 증거구별 · 내 점수 ___ / 2

정답 · 채점 기준

손실이라고 단정할 수 없음 (1점). 미push 또는 다른 브랜치에 게시 여부와 원격·브랜치 확인 (1점).

해설

로컬 존재와 특정 원격 브랜치 반영 상태를 나누어 본다.

근거 5장 p.14, 31-33; 5장 실습 p.10-14

05회 정답·해설

선택 적용·복구·원격 종합 | 4 / 5

70 | 통합명령 · 내 점수 ___ / 2

정답 · 채점 기준

git switch main ; git cherry-pick H (1점). git push origin main (1점).

해설

선택 적용은 로컬 작업이며 원격 반영에는 push가 필요하다.

근거 5장 p.7, 31

71 | 조건반전 · 내 점수 ___ / 2

정답 · 채점 기준

항상 오류는 아님 (1점). 현재 브랜치의 추적 관계·통합 설정·상태에 따라 판단 (1점).

해설

강의의 미설정 상황을 모든 저장소의 일반 규칙으로 외우지 않는다.

근거 5장 p.29, 31; 치트 p.1

72 | 상태비교 · 내 점수 ___ / 2

정답 · 채점 기준

soft: B 유지 (1점). hard: A로 복원 (1점).

해설

HEAD가 같은 A로 이동해도 index·작업트리에 미치는 범위가 다르다.

근거 치트 p.1

05회 정답·해설

선택 적용·복구·원격 종합 | 5 / 5

73 | PR시나리오 · 내 점수 ___ / 2

정답 · 채점 기준

같은 원격 feature의 새 F가 PR 변경에 포함 (1점). 로컬에만 있으면 서버 PR은 그 커밋을 아직 볼 수 없음 (1점).

해설

검토 대상은 원격 브랜치 상태다. 문제는 해당 브랜치에서 열린 일반 PR을 전제한다.

근거 5장 p.12, 33; 5장 실습 p.12-14 기반 응용

74 | 범위판단 · 내 점수 ___ / 2

정답 · 채점 기준

원본 feature의 끝 C와 기존 B 이력은 유지 (1점). main에 B의 변경을 적용한 새 기록을 만듦 (1점).

해설

선택 적용은 원본 커밋을 잘라 다른 곳에 옮기는 파일 이동이 아니다.

근거 5장 p.5-7; 보완 C

75 | 최종설계 · 내 점수 ___ / 2

정답 · 채점 기준

커밋 범위·의존성 확인 후 release에 cherry-pick하고 시험 (1점). 적절한 원격 브랜치에 push해 검토·통합 결과를 공유 (1점).

해설

명령 실행 성공, 코드의 기능 검증, 협업자에게 전달되는 상태를 모두 연결한다.

근거 5장 p.3, 7, 31, 39; 5장 실습 p.12-15

근거 안내 · 채점 보완

페이지는 PDF의 첫 장부터 센 실제 페이지입니다. “기반 응용”은 강의 개념에 새 조건을 더한 문항이며, 새 조건은 문제에 명시했습니다.

5장

20261006[오픈소스SW의_이해]CHAPTER05.pdf

5장 실습

20261006[오픈소스SW의_이해]CHAPTER05_실습.pdf

치트

Git_Cheat_Sheet_KO (1).pdf

명령의 전제

작업트리는 별도 설명이 없으면 깨끗하며, 인증·권한·명령 실행 환경은 정상입니다. A→B는 B의 부모가 A라는 뜻입니다. 상태 비교 문항에서 A/B/C는 파일 내용 표기입니다.

출제 범위 구분

1~3회는 강의의 선택 적용·원격·협업을 중심으로, 4회는 첨부 치트시트 확장, 5회는 종합 응용으로 구성했습니다. 추가 명령의 범위를 구분하며 학습할 수 있도록 확장 문항에 표시했습니다.

보완 C · 선택 적용

Cherry-pick은 커밋이 도입한 변경을 현재 이력 위에 적용합니다. 대상 브랜치 전체를 복사하지 않습니다. 충돌 해결 뒤 --continue, 취소 시 --abort를 사용합니다.

공식 보완 자료

Git: https://git-scm.com/docs/git-cherry-pick
https://git-scm.com/docs/git-reset
https://git-scm.com/docs/git-restore
https://git-scm.com/docs/git-stash
GitHub PR: https://docs.github.com/en/pull-requests/reference/pull-requests